

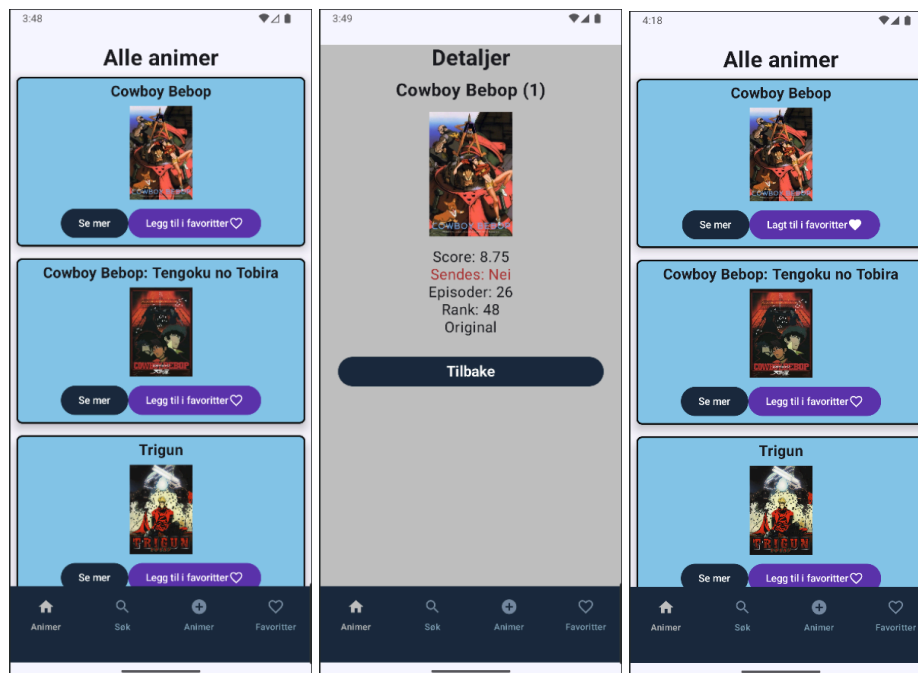
Eksamen Android programmering H25

Emnekode: PGR208
Emnenavn: Android programmering

Del 1. Redegjørelse av app per skjerm

Skjerm 1 - AnimesScreen + AnimeDetailScreen

På AnimeScreen vises alle animer fra Jikan API. Brukeren kan navigere til detaljside for hvert anime-objekt og legge de til i favoritter. Screen tar imot viewmodel og navcontroller. AnimesViewModel kommuniserer med både AnimeRepository for kontakt med API og AnimeFavoriteDbRepository for kontakt med databasen. Navcontroller brukes for å navigere til detaljside om hvert anime objekt.



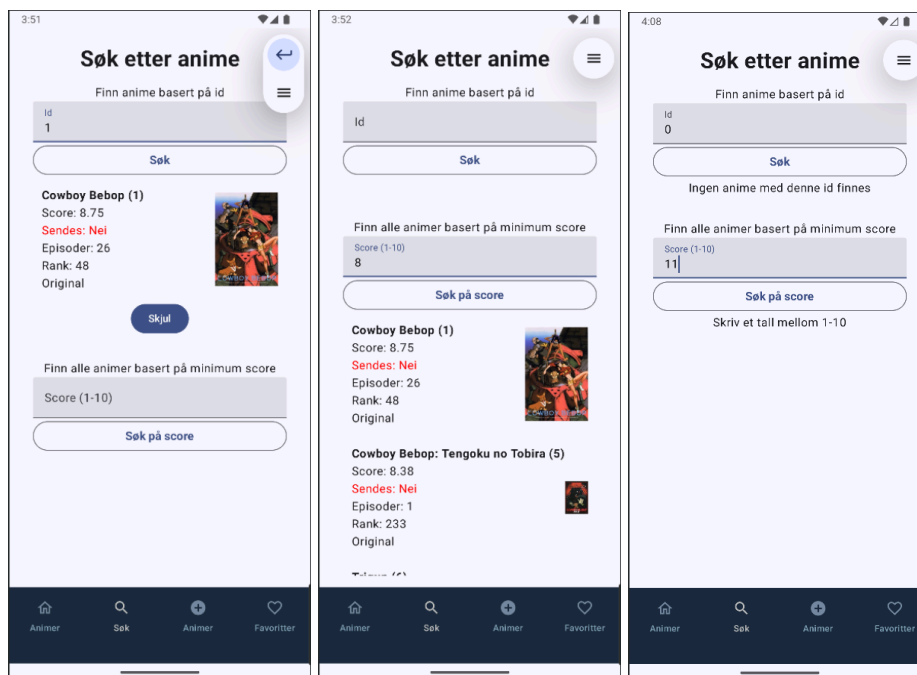
Gruppenummer: x, Kandidatnummere x, x, x

Når siden lastes inn utføres init-blokken i viewmodel som kjører funksjonen setAnimes. Denne henter alle animeer fra Jikan API gjennom AnimeRepository. I repository starter vi med å opprette en http-klient og et retrofit objekt som bruker baseUrl fra Jikan API. Vi har en suspend funksjon getAllAnimes i repository som bruker baseUrl sammen med endepunkt i interfacet. Dette gjør at vi får returnert alle animer fra Jikan API-et basert på endepunktet. Repository returnerer resultatet til viewmodel som lagres i en MutableStateFlow. Staten i viewmodel blir fylt med resultatet av API-kallet i en privat variabel og Screen benytter en collectAsState gjennom en offentlig variabel i viewmodel som speiler den private variabelen med listen av resultatet fra API-et Ved å benytte en privat variabel kan ikke listen med data endres fra andre steder enn i Viewmodel. Dataene som ligger i variabelen i screen kan dermed brukes til å opprette items som vises i grensesnittet. Vi benytter en lazyColumn for å vise AnimeListItems, fordi det er usikkert hvor mange objekter som hentes og listen lager kun objekter som vi ser på skjermen.

Når bruker trykker på “se mer” knappen blir de tatt til en detaljside om animen. Dette gjøres ved å navigere til en composable i NavHost som har ruten til AnimeDetailScreen. Det benyttes også en backStackEntry for å kunne gå tilbake. AnimeDetailScreen tar i mot en id som brukes til å hente inn riktig objekt gjennom viewmodels loadAnime metode.

Når bruker trykker på “legg til som favoritt” knappen blir animen lagt til tabellen animeFavorites i databasen. Det benyttes en insert metode der det opprettes et AnimFavorite-objekt av dataene fra Anime-objektet fra API. Dette gjorde vi for å gi favorittene samme id i databasen som i API.

Skjerm 2- AnimeSearchScreen



AnimeSearchScreen lar brukeren søke etter animer fra API-et på Id og minimumscore. Dette gjør man ved å skrive inn en verdi i tekstfeltet for id og score. Hvis man skriver en verdi som ikke er gyldig (for eksempel bokstav eller score over 10), kommer det ikke opp noe resultat, men en melding til bruker om hva som er galt. Når man har fylt inn en gyldig verdi, vil man se anime-objektet som matcher verdien, hentet ut fra JikanAPI og alle animer med høyere score.

AnimeSearchViewModel har metoder for å hente anime på id og hente liste med animer med score høyere enn angitt. Gjennom AnimeRepository mottas resultatene fra en GET i interfacet som har en Path for å lokalisere anime med en gitt id. For å hente ut liste med animer som har en score høyere enn verdien brukeren sender inn, bruker vi en query for å definere endepunktet i interfacet. Vi returnerer her en liste med Animes-objekter som har høyere score enn min-score verdien. Vi leste i dokumentasjonen for Jikan api for å se etter andre data som var tilgjengelig å hente ut og syntes det var en nyttig funksjon å legge til. Vi var innom andre queries som "order_by" og sort, men fant det hensiktsmessig at brukerne selv kunne skrive inn en verdi. Vi må bruke Query for å hente på min score da Jikan api krever et parameter med en tallverdi (anime?min_score=8) i motsetning til å hente på id der vi bruker path hvor filstien setter inn et tall etter / (anime/1).

I viewmodel brukes derfor to ulike MutableStateFlow for å holde på både anime etter id og en liste med animer basert på min score. Viewmodel inneholder også clear-metoder for å nullstille søkene til bruker ved bytting av sider. I screen er det to CollectAsStates som brukes til å lage AnimeSearchItems for å vise dataene fra API.

<https://docs.api.jikan.moe/#/anime/getanimesearch>

https://api.jikan.moe/v4/anime?min_score=8

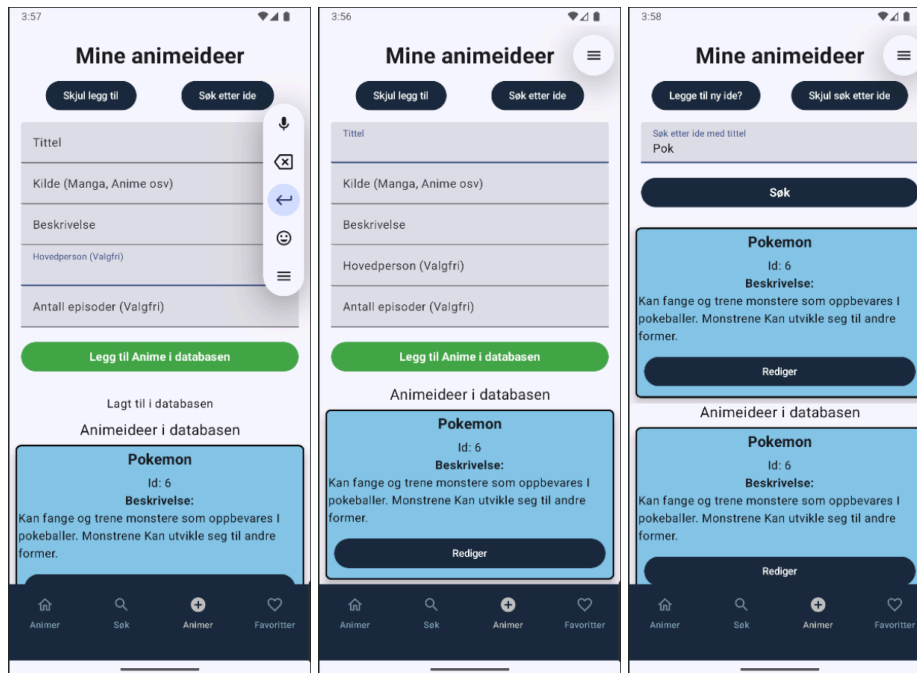
På denne siden er AnimeSearchViewModel ansvarlig for å administrere søkefunksjonaliteten, ved hjelp av funksjonene setAnimeById og setAnimeByScore. Her brukes StateFlow for å holde på og oppdatere søkeresultatene, som observeres med collectAsStateFlow, etterfulgt av å vise dem i UI.

Her bruker vi API-kall for å hente ut informasjon fra JikanAPI. Dette starter med AnimeRepository som bruker Retrofit til å håndtere http-kallene mot API-et. Her konfigureres OkHttpClient med en HttpLoggingInterceptor for å logge nettverksaktivitet og setter opp en Retrofit med baseUrl til nettsiden. Deretter konverteres JSON til Kotlin-objekter.

Ved hjelp av AnimeSearchViewModel får AnimeSearchScreen tilgang til AnimeRepository. Her brukes viewModelScope for å utføre API-kall asynkront, og oppdaterer objekter med resultatene. UI blir oppdatert ved at vi bruker setAnimeById og setAnimeByScore, som speiler getter-metodene i AnimeRepository og dermed henter og oppdaterer tilstanden til skjermen.

AnimeSearchScreen bruker LaunchedEffect(Unit) for å nullstille søkene når siden lastes inn. Det skjer ved at animeSearchViewModel.clearAnime() og animeSearchViewModel.clearAnimeByScore() blir kalt. Vi benytter en query for å hente animer basert på minScore. Dette oppnås ved at setAnimesByScore i AnimeSearchViewModel utfører et API-kall via AnimeRepository.getAnimesByScore("min_score"), som sender API-et med query-parameteren.

Skjerm 3 - AnimeIdeaScreen + AnimeUpdateScreen



I AnimeIdeasScreen gjør vi det mulig for brukeren å legge til og søke på sine egne animeideer. I brukergrensesnittet har vi lagt inn flere TextFields for å gi brukeren mulighet til å skrive inn de ulike egenskapene som til sammen utgjør et objekt. Skjermen er koblet opp til databasen og har sin egen tabell, der vi oppbevarer data for animeideer. Skjermen viser også ideer som er i databasen, og det er mulig for brukeren å navigere seg inn på en ny side for å redigere eller slette denne animeideen.

Vi har brukt en if-else-løkke, slik at feltene man bruker for å skrive inn properties til en ny anime-idé vil fjernes automatisk hvis man trykker på knappen for å søke etter anime-idéer, slik at det blir mer intuitivt for brukeren. I tillegg har man mulighet til å skjule begge fanene ved å klikke på skjul: legg til ny idé/søk etter idé. Disse knappene endrer innhold avhengig av hvilke faner på skjermen som er synlige. Hovedperson og antall episode er valgfrie å fylle inn når man skal lage en ny idé.

I likhet med de andre skjermene, tar denne siden imot en ViewModel og NavController, henholdsvis for å holde på data med objektene som skapes og for å kunne navigere seg lett mellom de ulike skjermene. Vi benytter oss av Room-database med AnimeIdeaDbRepository, for å enkelt holde på tilstanden til objektene og endre dem om brukeren velger å gjøre det. Dette har vi kombinert med MutableStateFlow, slik at brukergrensesnittet oppdaterer seg dynamisk sammen med databasen.

Items i denne skjermen, med andre ord AnimeIdeaListItem, er det som vises når man lager nye anime-objekter. Her har vi brukt LazyColumn for å være sikre på at alle de nye anime-ideene som blir skapt, blir vist på skjermen.

Vi har lagt inn sjekker for å hindre at bruker kan legge til ufullstendige objekter. Her har vi blant annet brukt isNotBlank på feltene som er helt nødvendig å fylle ut. Enkelte felter behøver ikke å fylles ut, men kan legges til i ettertid. Vi benytter "?" og satt en standard verdi for disse variablene i AnimeIdea entiteten for gjøre dette mulig.

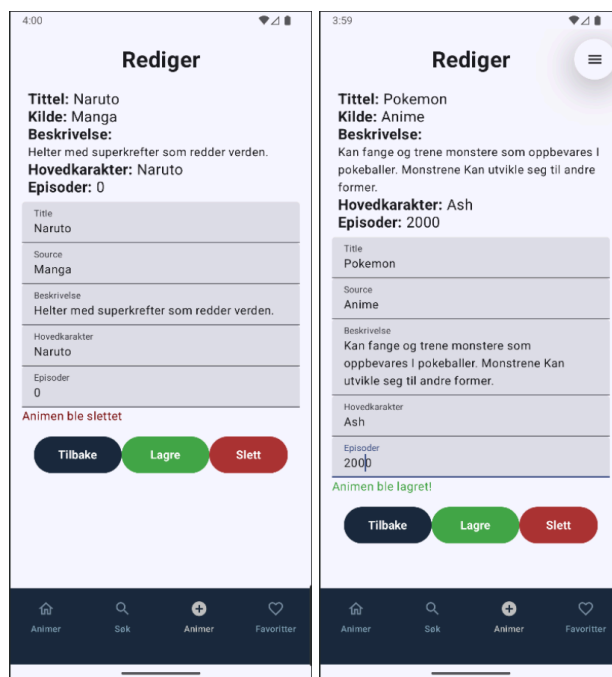
Gruppenummer: x, Kandidatnummere x, x, x

Søkfunksjonen fungerer ved at vi har en sql spørring i dao, som henter alle animer i tabellen som inneholder ordet som skrives inn. Her benytter en LIKE i spørringen og sender inn et parameter med %-tegn på begge sider, slik at verdiene både før og etter får treff hvis ordet matcher.

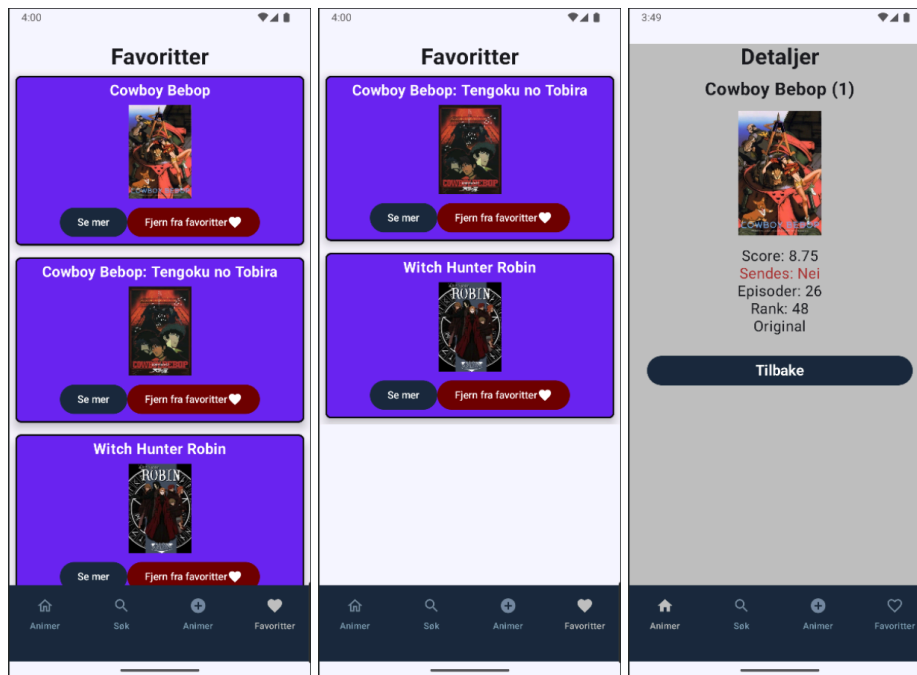
UpdateScreen:

Dette er en skjerm man navigerer seg til gjennom idé-siden ved å trykke på rediger på et anime-item. Her får man opp de ulike feltene som må fylles inn for å endre på propertiene til et item som allerede eksisterer databasen. Over feltene får man se de nåværende verdiene og under har man en lagre-, slett-, og tilbake-knapp.

det benyttes en update i dao som tar i mot et AnimeIdea objekt.



Skjerm 4 - AnimeFavoriteScreen + AnimeDetailScreen



På skjerm 4 - FavoriteScreen - har brukeren oversikt over sine favorittanimeer. I screen kjøres en LaunchedEffekt for å sette animeFavorites variabelen og fylle den med objekter fra databasen. Viewmodel kaller på AnimeFavoriteDbRepository sin getAllAnimes som kjører en SELECT-spørring i Dao mot databasen. Resultatet fra spørringen sendes til repository og MutableStateFlowen i viewmodel fylles med innholdet fra databasen.

Brukeren kan trykke på “se mer” og komme inn på detaljsiden om animen. Dette er samme side som man også kommer til fra skjerm 1. AnimeDetailsScreen tar i mot en id for å vise en bestemt anime. Vi benytter samme id for objektene i databasen som i API, for å kunne hente direkte fra Jikan API.

Det er også en “fjern fra favoritt” knapp som benytter en Delete i dao. Den tar i mot en id og sletter objektet med denne id i tabellen. Knappen trigger også en funksjon som henter inn gjenværende objekter fra databasen på nytt slik at det slettede objektet forsvinner fra grensesnittet. AnimeFavoriteItem tar i mot en metode, removeFavorite, som tar i mot selve objektet som itemet lages av og sender med dette via viewmodel for å slette objektet fra databasen. Den tar også i mot en showAnime metode som bruker NavigationRoutes til å navigere til den bestemte animens detaljside basert på id.

Del 2. Redegjørelse av planlegging og utvikling

Hvordan har vi planlagt løsningen?

Vi skaffet oss først et godt overblikk over hvordan Jikan APIet var strukturert, for å skjønne hvordan vi måtte bygge opp løsningen, med tanke på ID og andre instansvariabler. Deretter, satt vi opp filstrukturen, slik at vi fikk en god oversikt over hvordan vi skulle bygge opp løsningen, med ekstra fokus på hvilke funksjoner appen skulle ha, og dermed hvilke skjermer og tilhørende ViewModels vi måtte inkludere. Utover dette var vi bevisst på å dele opp i flere repositories for å dekke kravet om separation of concerns.

Hvordan har vi lagt opp arbeidet og fordelingen av arbeidet?

Gruppearbeidet har hovedsakelig foregått online, der vi har fordelt oppgaver, som vi har løst individuelt, og deretter sett gjennom i plenum. På de mer krevende delene har en av oss delt skjerm, slik at vi har samarbeidet på samme løsning i sanntid, der vi også har byttet på hvem som har tatt seg av kodingen i dette fellesarbeidet. De mindre krevende problemene, som for eksempel å lage nav-bar eller lage ulike screens, har vi fordelt på alle tre og løst individuelt, etterfulgt av å sette alt sammen under møtene.

Hvordan har vi kommet fram til avgjørelsene?

Vi diskuterte det aller meste over online møter, slik at vi var sikre på at alle var oppdatert på de siste endringene og hadde full oversikt over hvilke endringer som ble gjort på løsningen. Vi lastet ned hver vår versjon av løsningen og gjorde små endringer, men tok det opp i plenum for å avgjøre om vi ville ha med endringene videre eller ei.

Hvordan gikk samarbeidet?

Vi synes det gikk bra, siden vi hadde en god kommunikasjon og alle var oppdatert på de siste versjonene av koden vi jobbet med.